

1. Chasis (**DriveTrain**)

This class controls the robot's tank-like chassis, which consists of two motors (one right and one left). The constructor retrieves references to the motors from the HardwareMap, configures the braking behavior (BRAKE) so the robot stops abruptly instead of skidding when power is lost, and reverses the direction of the left motor so both motors rotate coherently when receiving the same signal. The drive() method combines the forward (speed) and turn (turn) values to calculate the power for each motor, allowing the robot to move forward, backward, and in combined turns.

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.hardware.DcMotorEx;
import com.qualcomm.robotcore.hardware.DcMotorSimple;
import com.qualcomm.robotcore.hardware.HardwareMap;

public class DriveTrain {

    DcMotorEx rightMotor;
    DcMotorEx leftMotor;

    HardwareMap hw;

    public DriveTrain(HardwareMap hw){

        this.hw = hw;
        rightMotor = hw.get(DcMotorEx.class, "rightMotor");
        leftMotor = hw.get(DcMotorEx.class, "leftMotor");

        rightMotor.setZeroPowerBehavior(DcMotorEx.ZeroPowerBehavior.BRAKE);
        leftMotor.setZeroPowerBehavior(DcMotorEx.ZeroPowerBehavior.BRAKE);

        leftMotor.setDirection(DcMotorSimple.Direction.REVERSE);
    }

    public void drive(double speed, double turn){
        double rightPower = speed - turn;
        double leftPower = speed + turn;

        rightMotor.setPower(rightPower);
        leftMotor.setPower(leftPower);

    }
}
```

```
}
```

2. Subsystems (Subsystems)

This class groups the robot's mechanisms that are not part of the chassis: the climber motor and the intake motor. In the constructor, both motors are obtained from the HardwareMap, and the climber's automatic brake is activated to maintain its position when it is not receiving power. The `setPowerClimber()` and `setPowerIntake()` methods allow the power of each motor to be controlled independently from the TeleOp.

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.hardware.DcMotorEx;
import com.qualcomm.robotcore.hardware.HardwareMap;

public class Subsystems {

    DcMotorEx climber;
    DcMotorEx intake;

    HardwareMap hw;

    public Subsystems(HardwareMap hw){

        this.hw = hw;
        climber = hw.get(DcMotorEx.class, "climber");
        climber.setZeroPowerBehavior(DcMotorEx.ZeroPowerBehavior.BRAKE);

        intake = hw.get(DcMotorEx.class, "intake");
    }

    public void setPowerClimber(double power){
        climber.setPower(power);
    }

    public void setPowerIntake(double power){
        intake.setPower(power);
    }
}
```

Nota: en el documento original a esta clase le falta la llave `}` de cierre. La dejé tal como la compartiste, sin corregirla, en caso de que sea solo un recorte al copiar el texto.

3. TeleOp (MyTeleOp)

3.1 Declaración de la clase y variables miembro

The MyTeleOp class extends LinearOpMode, which is the type of program that runs during the manual control (TeleOp) period of an FTC match. Two member variables, `m_drive` and `m_subsystems`, are declared to store the chassis and subsystem instances for use throughout `runOpMode()`.

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
import com.qualcomm.robotcore.eventloop.opmode.Disabled;
import com.qualcomm.robotcore.hardware.DcMotor;
import com.qualcomm.robotcore.hardware.DcMotorSimple;
import com.qualcomm.robotcore.util.ElapsedTime;

@TeleOp
public class MyTeleOp extends LinearOpMode {

    private DriveTrain m_drive;
    private Subsystems m_subsystems;
```

3.2 Inicialización del robot

When OpMode starts, instances of DriveTrain and Subsystems are created, passing them the robot's hardware map so that each class can locate its own motors according to the Control Hub configuration. A telemetry message is then sent indicating that the robot has finished initializing, and the program pauses at `waitForStart()` until the operator presses PLAY on the Driver Station.

```
@Override
public void runOpMode() {

    m_drive = new DriveTrain(hardwareMap);
    m_subsystems = new Subsystems(hardwareMap);
```

```
telemetry.addData("Status", "Initialized");
telemetry.update();
// Wait for the game to start (driver presses PLAY)
telemetry.update();
waitForStart();
```

Note: `telemetry.update();` appears to be a typo (you probably meant to write `telemetry.update();`). I left it as it is in your document, since you only asked me to add explanations, but I'm pointing it out in case you want to correct it.

3.3 Bucle principal y manejo del chasis

Once the match starts, the program enters a loop that repeats while OpMode is active (`opModeIsActive()`). On each turn, the left stick (forward/backward) and right stick (turn) inputs of gamepad 1 are read, inverting the sign because on most gamepads the Y-axis returns negative values when pushing forward. These values are sent to the chassis's `drive()` method to move the robot.

```
// run until the end of the match (driver presses STOP)

while (opModeIsActive()) {

    m_drive.drive(-gamepad1.left_stick_y, -gamepad1.right_stick_x);
```

3.4 Lectura de botones

Boolean variables are declared here to read the state of the cross (D-Pad up/down) and the left/right bumpers of the gamepad 1. These variables act as flags that indicate, on each turn of the loop, whether the operator is pressing each button at that moment.

```
boolean DPadUp = false;
boolean DPadDown = false;
boolean isLeftBumpPressed = false;
boolean isRightBumpPressed = false;

DPadUp = gamepad1.dpad_up;
DPadDown = gamepad1.dpad_down;
isLeftBumpPressed = gamepad1.left_bumper;
isRightBumpPressed = gamepad1.right_bumper;
```

3.5 Control del intake

With the state of the bumpers already read, this block decides the power of the intake: if the left bumper is pressed, the intake rotates inwards (power 1); if the right one is pressed, it rotates outwards (power -1); and if neither is pressed, the intake stops (power 0).

```
if(isLeftBumpPressed){
    m_subsystems.setPowerIntake(1);
} else if(isRightBumpPressed){
    m_subsystems.setPowerIntake(-1);
}
else{
    m_subsystems.setPowerIntake(0);
}
```

3.6 Control del escalador

Similarly, this block uses the D-pad to control the climber: D-pad up raises it (power 1), D-pad down lowers it (power -1), and if neither is pressed, the motor stops (power 0). Since the climber's motor is set to automatic braking (BRAKE), at power 0 it will maintain its position instead of falling freely.

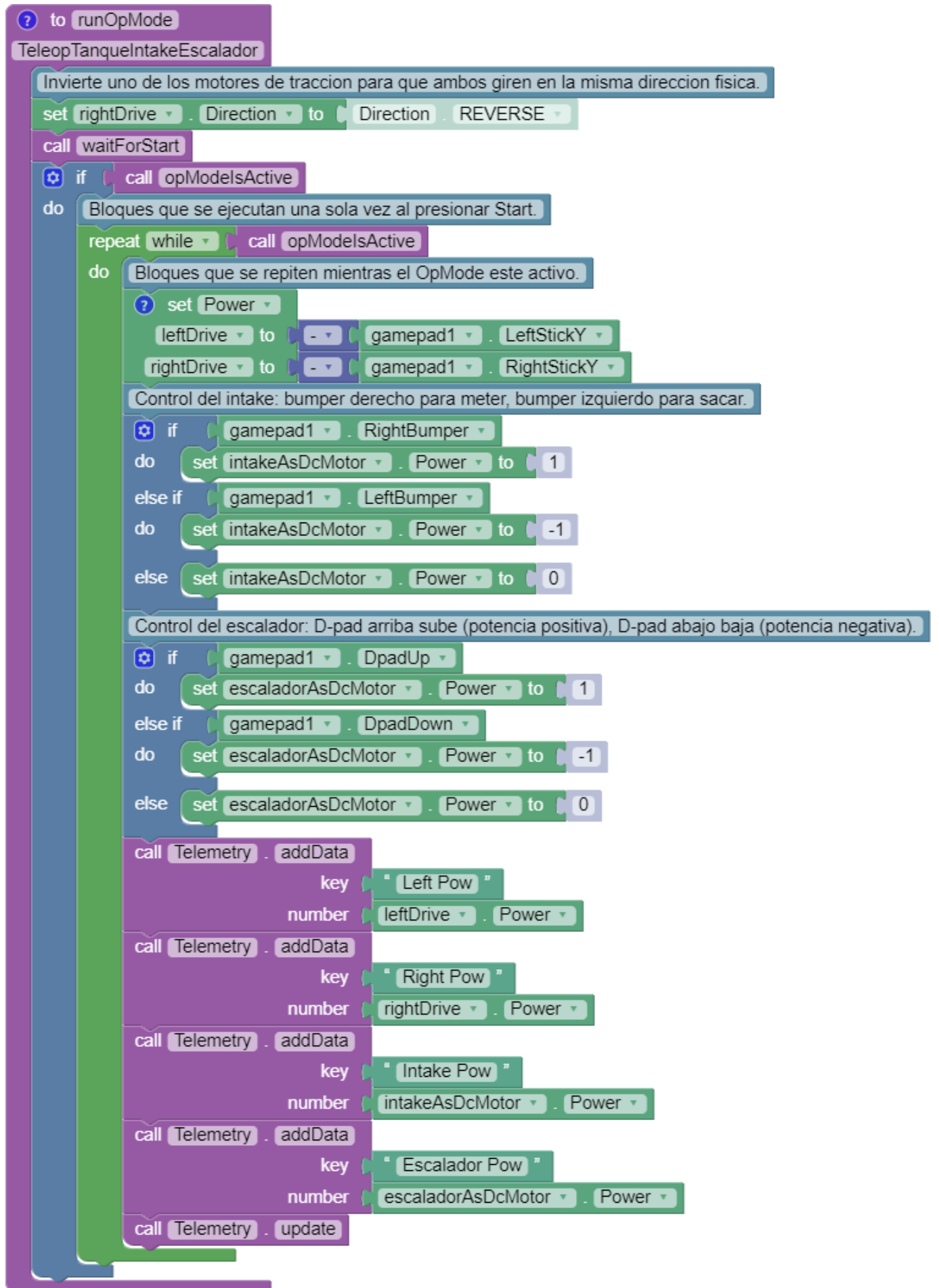
```
if(DPadUp){
    m_subsystems.setPowerClimber(1);
} else if(DPadDown){
    m_subsystems.setPowerClimber(-1);
}
else{
    m_subsystems.setPowerClimber(0);
}

}

}
```

4.0 Blocks

This FTC code project is organized into three parts: the DriveTrain class controls the robot's tank-style chassis, made up of two motors (right and left), where the constructor gets the motor references from the HardwareMap, sets automatic braking (BRAKE) so the robot stops sharply instead of coasting, and reverses the direction of the left motor so both motors spin consistently, while the drive() method combines the forward value (speed) and the turn value (turn) to calculate the power for each motor, moving the robot forward, backward, and through combined turns; the Subsystems class groups the robot's additional mechanisms — the climber motor and the intake motor — getting both from the HardwareMap in the constructor and enabling automatic braking on the climber so it holds its position when it isn't receiving power, while the setPowerClimber() and setPowerIntake() methods allow each motor's power to be controlled independently; and finally the MyTeleOp class, which extends LinearOpMode and is the program that runs during the match's manual-control period, declares the m_drive and m_subsystems variables to hold the chassis and subsystems instances, initializes them with the hardwareMap when the OpMode starts, sends a status telemetry message, and waits with waitForStart() for the operator to press PLAY, and once the match begins enters a loop that, while the OpMode is active, reads gamepad 1's sticks to drive the chassis, reads the D-Pad and bumpers to know which buttons are pressed, controls the intake so it spins inward with the left bumper, outward with the right bumper, and stops if neither is pressed, and controls the climber similarly using the D-Pad to raise it, lower it, or hold it in place thanks to the automatic brake configured on its motor.



4.1 Copy and paste

The next code you can copy and pasted on Blocks and is going to appear like blocks:

```
<xml xmlns="https://developers.google.com/blockly/xml">
<block type="procedures_defnoreturn" id="713465f8abab4ccda6d8" deletable="false" x="25" y="133">
<field name="NAME">runOpMode</field>
<comment pinned="false" h="106" w="552">This OpMode offers Tank Drive style TeleOp control with
speed/turn mixing for a direct drive robot, plus intake control on the bumpers and climber control on the
D-Pad.</comment>
<data>{"commentPositionLeft":-20,"commentPositionTop":-133}</data>
<statement name="STACK">
<block type="comment" id="07e72d34d78d4be0bd4d">
<field name="COMMENT">Reverse one of the drive motors.</field>
<next>
<block type="dcMotor_setProperty_Direction" id="289029c8121f4813869c">
<field name="IDENTIFIER">leftMotorAsDcMotor</field>
<field name="PROP">Direction</field>
<data>{"IDENTIFIER":"leftMotor"}</data>
<value name="VALUE">
<shadow type="dcMotor_typedEnum_direction" id="b1cc387552074cc69fc1">
<field name="DIRECTION">REVERSE</field>
</shadow>
</value>
<next>
<block type="dcMotor_setProperty_ZeroPowerBehavior" id="d2c9e501ba0f46808b63">
<field name="IDENTIFIER">rightMotorAsDcMotor</field>
<field name="PROP">ZeroPowerBehavior</field>
<data>{"IDENTIFIER":"rightMotor"}</data>
<value name="VALUE">
<shadow type="dcMotor_typedEnum_zeroPowerBehavior" id="9d222987d6b54e508090">
<field name="ZERO_POWER_BEHAVIOR">BRAKE</field>
</shadow>
</value>
<next>
<block type="dcMotor_setProperty_ZeroPowerBehavior" id="5b17b854f14443c79a17">
<field name="IDENTIFIER">leftMotorAsDcMotor</field>
<field name="PROP">ZeroPowerBehavior</field>
<data>{"IDENTIFIER":"leftMotor"}</data>
<value name="VALUE">
<shadow type="dcMotor_typedEnum_zeroPowerBehavior" id="19486aca41224b7dace1">
<field name="ZERO_POWER_BEHAVIOR">BRAKE</field>
```

```

</shadow>
</value>
<next>
<block type="dcMotor_setProperty_ZeroPowerBehavior" id="a31f1e6f70dd424fabdc">
<field name="IDENTIFIER">climberAsDcMotor</field>
<field name="PROP">ZeroPowerBehavior</field>
<data>{"IDENTIFIER":"climber"}</data>
<value name="VALUE">
<shadow type="dcMotor_typedEnum_zeroPowerBehavior" id="4bb5cc93939a45ddadfa">
<field name="ZERO_POWER_BEHAVIOR">BRAKE</field>
</shadow>
</value>
<next>
<block type="linearOpMode_waitForStart" id="cc45c0452a87487fb58b">
<next>
<block type="controls_if" id="3f3137ed94324c4390f0">
<value name="IF0">
<block type="linearOpMode_opModelsActive" id="6444ffbd08c14625b71f" />
</value>
<statement name="DO0">
<block type="comment" id="976af50d8580424b9958">
<field name="COMMENT">Put run blocks here.</field>
<next>
<block type="controls_whileUntil" id="0d4ed7c4cb3542a6b286">
<field name="MODE">WHILE</field>
<value name="BOOL">
<block type="linearOpMode_opModelsActive" id="7bd56e0064764e3eb9e3" />
</value>
<statement name="DO">
<block type="dcMotor_setProperty_Power" id="ac85da80c1634aa6a19f">
<field name="IDENTIFIER">rightMotorAsDcMotor</field>
<field name="PROP">Power</field>
<data>{"IDENTIFIER":"rightMotor"}</data>
<value name="VALUE">
<shadow type="math_number" id="9060da4162e94228a6d4">
<field name="NUM">1</field>
</shadow>
<block type="math_arithmetic" id="bf8adf63ae6f47c7a1dd">
<field name="OP">MINUS</field>
<value name="A">
<shadow type="math_number" id="6a7219be9b5e4b7fbc1d">
<field name="NUM">1</field>
</shadow>

```

```
<block type="math_single" id="aa1993d6d1554fe3af70">
  <field name="OP">NEG</field>
  <value name="NUM">
    <block type="gamepad_getProperty_Number" id="321cbcf048b64a6aae12">
      <field name="IDENTIFIER">gamepad1</field>
      <field name="PROP">LeftStickY</field>
      <data>{"IDENTIFIER":"gamepad1"}</data>
    </block>
  </value>
</block>
</value>
<value name="B">
  <shadow type="math_number" id="4ec86b41bc24467aa670">
    <field name="NUM">1</field>
  </shadow>
  <block type="math_single" id="7629b67cac03431ea7bc">
    <field name="OP">NEG</field>
    <value name="NUM">
      <block type="gamepad_getProperty_Number" id="64067c331a73427aa78a">
        <field name="IDENTIFIER">gamepad1</field>
        <field name="PROP">RightStickX</field>
        <data>{"IDENTIFIER":"gamepad1"}</data>
      </block>
    </value>
  </block>
</value>
</block>
</value>
<next>
  <block type="dcMotor_setProperty_Power" id="86ac532ca1dc4de2b9bc">
    <field name="IDENTIFIER">leftMotorAsDcMotor</field>
    <field name="PROP">Power</field>
    <data>{"IDENTIFIER":"leftMotor"}</data>
    <value name="VALUE">
      <shadow type="math_number" id="b172673f88fa4bba9537">
        <field name="NUM">1</field>
      </shadow>
      <block type="math_arithmetic" id="c8b2e20bcedc4e488f52">
        <field name="OP">ADD</field>
        <value name="A">
          <shadow type="math_number" id="1b0b6f3ed88347df94d2">
            <field name="NUM">1</field>
          </shadow>
```

```
<block type="math_single" id="8691611021d04418bec9">
  <field name="OP">NEG</field>
  <value name="NUM">
    <block type="gamepad_getProperty_Number" id="74cd2e6f41ff4e2099a6">
      <field name="IDENTIFIER">gamepad1</field>
      <field name="PROP">LeftStickY</field>
      <data>{"IDENTIFIER":"gamepad1"}</data>
    </block>
  </value>
</block>
</value>
<value name="B">
  <shadow type="math_number" id="4559ba53cfe841f0bb79">
    <field name="NUM">1</field>
  </shadow>
  <block type="math_single" id="5b152a36397b48dbab51">
    <field name="OP">NEG</field>
    <value name="NUM">
      <block type="gamepad_getProperty_Number" id="621fd2dcd48e42a3bcf4">
        <field name="IDENTIFIER">gamepad1</field>
        <field name="PROP">RightStickX</field>
        <data>{"IDENTIFIER":"gamepad1"}</data>
      </block>
    </value>
  </block>
</value>
</block>
</value>
<next>
  <block type="controls_if" id="e9577f3506fb4be9ae77">
    <mutation elseif="1" else="1" />
    <value name="IF0">
      <block type="gamepad_getProperty_Boolean" id="2261179ad2034ed0bb83">
        <field name="IDENTIFIER">gamepad1</field>
        <field name="PROP">LeftBumper</field>
        <data>{"IDENTIFIER":"gamepad1"}</data>
      </block>
    </value>
    <statement name="DO0">
      <block type="dcMotor_setProperty_Power" id="3d483a2e7fa24024948d">
        <field name="IDENTIFIER">intakeAsDcMotor</field>
        <field name="PROP">Power</field>
        <data>{"IDENTIFIER":"intake"}</data>
```

```

<value name="VALUE">
<shadow type="math_number" id="401c29cefccd4f09b648">
<field name="NUM">1</field>
</shadow>
</value>
</block>
</statement>
<value name="IF1">
<block type="gamepad_getProperty_Boolean" id="1335e8b98cb64583bbd0">
<field name="IDENTIFIER">gamepad1</field>
<field name="PROP">RightBumper</field>
<data>{"IDENTIFIER": "gamepad1"}</data>
</block>
</value>
<statement name="DO1">
<block type="dcMotor_setProperty_Power" id="58956084ed494304beca">
<field name="IDENTIFIER">intakeAsDcMotor</field>
<field name="PROP">Power</field>
<data>{"IDENTIFIER": "intake"}</data>
<value name="VALUE">
<shadow type="math_number" id="cd9b961aa68344fbba8d">
<field name="NUM">-1</field>
</shadow>
</value>
</block>
</statement>
<statement name="ELSE">
<block type="dcMotor_setProperty_Power" id="e5cac4e9bfa3419facc3">
<field name="IDENTIFIER">intakeAsDcMotor</field>
<field name="PROP">Power</field>
<data>{"IDENTIFIER": "intake"}</data>
<value name="VALUE">
<shadow type="math_number" id="bec7de18ef2b46c7aed4">
<field name="NUM">0</field>
</shadow>
</value>
</block>
</statement>
<next>
<block type="controls_if" id="897e7ab755344dbeb1dc">
<mutation elseif="1" else="1" />
<value name="IF0">
<block type="gamepad_getProperty_Boolean" id="d518e2210c714b7fa747">

```

```
<field name="IDENTIFIER">gamepad1</field>
<field name="PROP">DpadUp</field>
<data>{"IDENTIFIER":"gamepad1"}</data>
</block>
</value>
<statement name="DO0">
<block type="dcMotor_setProperty_Power" id="ca3e6226c15e49b09070">
<field name="IDENTIFIER">climberAsDcMotor</field>
<field name="PROP">Power</field>
<data>{"IDENTIFIER":"climber"}</data>
<value name="VALUE">
<shadow type="math_number" id="54c3874fa1b14cb4ada2">
<field name="NUM">1</field>
</shadow>
</value>
</block>
</statement>
<value name="IF1">
<block type="gamepad_getProperty_Boolean" id="11cceee23cd94844b428">
<field name="IDENTIFIER">gamepad1</field>
<field name="PROP">DpadDown</field>
<data>{"IDENTIFIER":"gamepad1"}</data>
</block>
</value>
<statement name="DO1">
<block type="dcMotor_setProperty_Power" id="d4a8058f20be4eab802d">
<field name="IDENTIFIER">climberAsDcMotor</field>
<field name="PROP">Power</field>
<data>{"IDENTIFIER":"climber"}</data>
<value name="VALUE">
<shadow type="math_number" id="0588f8798eeb46e09486">
<field name="NUM">-1</field>
</shadow>
</value>
</block>
</statement>
<statement name="ELSE">
<block type="dcMotor_setProperty_Power" id="505da0f30fb7482ead78">
<field name="IDENTIFIER">climberAsDcMotor</field>
<field name="PROP">Power</field>
<data>{"IDENTIFIER":"climber"}</data>
<value name="VALUE">
<shadow type="math_number" id="753f37bb3a9f4c72bb87">
```

```
<field name="NUM">0</field>
</shadow>
</value>
</block>
</statement>
</block>
</next>
</block>
</next>
</block>
</next>
</block>
</statement>
</block>
</next>
</block>
</statement>
</block>
</next>
</block>
</next>
</block>
</next>
</block>
</next>
</block>
</next>
</block>
</next>
</block>
</next>
</block>
</statement>
</block>
</xml>
<?xml version='1.0' encoding='UTF-8' standalone='yes' ?>
<Extra>
<OpModeMeta flavor="TELEOP" group="" autoTransition="" />
<Enabled value="true" />
</Extra>
```